

Course Module: Generative Adversarial Networks (GANs)

Latreche Sara

Contents

1	Introduction to GANs	2
2	Historical Background of GANs	4
3	Mathematical Formulation	4
4	How do GANs work?	4
5	GANs in action	5
5.1	GAN training algorithm	7
5.2	Reaching Equilibrium	7
6	Training Tips and Challenges	9
7	Hands-on Lab: Implementing a Simple GAN in PyTorch	10
8	Variants of GANs	11
9	Applications of GANs	11

1 Introduction to GANs

The idea of whether machines can truly *think* is older than the computer itself. In 1950, the mathematician and logician **Alan Turing**, best known for helping crack the Nazi Enigma code, published a visionary paper: “*Computing Machinery and Intelligence*.”

In it, he introduced what is now called the **Turing Test**: if a machine can converse with a human so convincingly that the human cannot tell whether the counterpart is a machine or a person, then the machine must be deemed intelligent.

For decades, computers excelled at very different tasks—recognizing patterns, classifying data, even mastering complex games like chess or Go. Yet they consistently failed at what feels most natural to humans: **creativity and conversation**. Chatbots felt mechanical, computer-generated content looked artificial, and machines were unable to produce realistic new data beyond what they had already seen.

This all changed in **2014**. Ian Goodfellow, then a PhD student at the Université de Montréal under Yoshua Bengio, introduced the world to **Generative Adversarial Networks (GANs)**—a radical new approach to generation.

The story of their birth has become an AI legend: one evening, while discussing generative models with friends, Goodfellow rejected the idea of directly modeling probability distributions. Instead, he imagined a competition between two networks: one that tries to generate fake data and one that tries to detect it. That night, he coded the prototype on his laptop, and it worked.

As Goodfellow later joked: “*It was a Saturday night, and my friends wanted to go to a bar. Instead, I went home and coded up GANs.*”

With this invention, computers suddenly gained a new creative capacity. GANs could learn to generate **photorealistic faces**, transform **sketches into images**, create **artworks in specific styles**, and even produce **texts, poems, and music**. What had seemed nearly impossible—machines producing data indistinguishable from reality—was now within reach.



Figure 1: Progress in human face generation 2016-2017 (Source: “The Malicious Use of Artificial Intelligence: Forecasting, Prevention, and Mitigation,” by Miles Brundage et al., 2018, <https://arxiv.org/abs/1802.07228>.)

Definition

Generative Adversarial Networks (GANs) are a class of machine learning models that consist of two simultaneously trained neural networks:

- The **Generator (G)** is trained to produce synthetic data that resembles the real dataset.
- The **Discriminator (D)** is trained to distinguish between genuine examples from the dataset and fake examples created by the Generator.

The term *generative* highlights the goal of creating new data, while *adversarial* reflects the competitive dynamic between the two networks. The Generator aims to fool the Discriminator, while the Discriminator improves its ability to detect forgeries—much like an art forger trying to replicate da Vinci’s paintings and an expert assessing their authenticity.

This adversarial game drives both networks to improve simultaneously: the Generator learns to create increasingly convincing data, and the Discriminator becomes sharper at spotting fakes. Depending on the implementation, GANs can use simple feed-forward architectures, convolutional networks, or even advanced designs like U-Nets.

Active Recall – GANs Core Concept

Q: What is the adversarial aspect of GANs?

A: The generator tries to fool the discriminator while the discriminator tries to correctly classify real vs generated samples. They improve together through competition.

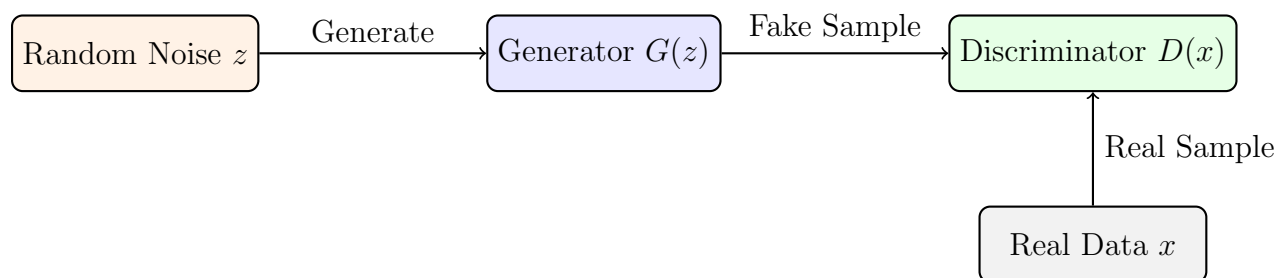


Figure 2: Basic architecture of a GAN: the generator produces fake data and the discriminator distinguishes between real and fake samples.

2 Historical Background of GANs

Example

GANs were introduced by **Ian Goodfellow et al. (2014)**. Key milestones:

- **2014:** Original GAN paper; first demonstration of adversarial training.
- **2015:** DCGANs (Deep Convolutional GANs) improve training stability and image quality.
- **2016–2018:** Conditional GANs, CycleGANs, and WGANs introduce new capabilities and loss functions.
- **2019–2022:** GANs applied to super-resolution, style transfer, and 3D generation.

Active Recall – GAN History

Q: What was the main contribution of DCGANs?

A: Introduced convolutional architectures to stabilize GAN training and produce higher-quality images.

3 Mathematical Formulation

4 How do GANs work?

GANs are built from two competing neural networks:

- The **Generator (G)** creates synthetic data from random noise, aiming to produce samples indistinguishable from real data.
- The **Discriminator (D)** acts as a classifier, learning to distinguish real data from the Generator's fake samples.

Their interaction drives both networks to improve simultaneously. Table 1 summarizes their roles.

	Generator (G)	Discriminator (D)
Input	Random noise vector	Real samples from dataset Fake samples from Generator
Output	Fake samples (synthetic data)	Probability that input is real
Goal	Generate realistic data that mimics the training set	Distinguish real data from fake data

Table 1: Summary of the Generator and Discriminator subnetworks.

5 GANs in action

Now that we have a high-level view of GANs, let's look at how the system operates in practice. The core idea is simple: the Generator creates synthetic samples, while the Discriminator judges them against real data. Both are iteratively trained to improve together.

Figure 3 shows the typical GAN architecture, where the Generator receives random noise as input and produces fake samples, while the Discriminator compares both real and fake samples and outputs the probability of authenticity.

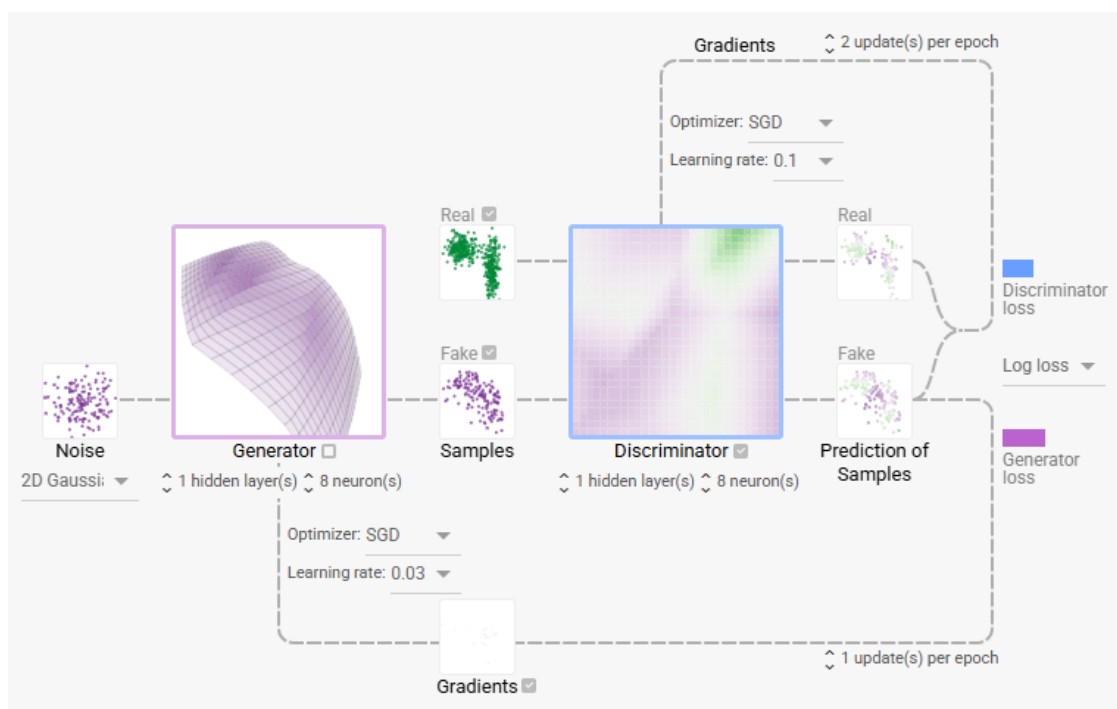


Figure 3: Interactive visualization of GAN training: the Generator creates samples from noise, while the Discriminator classifies real vs. fake. (Source: <https://poloclub.github.io/ganlab/>)

Walkthrough of the GANLab Architecture (Figure 3):

1. **Noise input (left):** The process starts with random noise, typically sampled from a Gaussian distribution. This serves as the raw material for generating data.
2. **Generator (purple box):** The Generator transforms the random noise into synthetic samples (“fake data”). Its goal is to produce outputs that resemble the real dataset as closely as possible.
3. **Samples (middle):** The generated fake data is compared side by side with real samples drawn from the dataset. Both types of data are passed to the Discriminator.
4. **Discriminator (blue box):** The Discriminator attempts to classify inputs as either “real” or “fake.” It outputs a probability score indicating authenticity.
5. **Training feedback (dashed arrows):**
 - If the Discriminator is fooled, the Generator receives positive reinforcement to continue in that direction.
 - If the Discriminator correctly rejects a fake, the Generator adjusts its parameters to improve.
 - At the same time, the Discriminator updates its own parameters to better distinguish real from fake.
6. **Losses (right):** The Generator loss (purple) decreases when fake samples look real; the Discriminator loss (blue) decreases when it correctly identifies authenticity. Both networks are updated iteratively, creating the adversarial dynamic.

5.1 GAN training algorithm

The training process alternates between improving the Discriminator and improving the Generator.

Algorithm 1 GAN Training Algorithm

- 1: **for** each training iteration **do**
 - 2: **Train Discriminator:**
 - a) Sample a real example x from the dataset.
 - b) Sample random noise z and generate a fake example x^* using the Generator.
 - c) Classify x and x^* with the Discriminator.
 - d) Update Discriminator parameters to minimize classification error.
 - 3: **Train Generator:**
 - a) Sample new random noise z and generate x^* .
 - b) Classify x^* with the Discriminator.
 - c) Update Generator parameters to maximize the Discriminator's error.
 - 4: **end for**
-

5.2 Reaching Equilibrium

Training a GAN is unlike training a standard neural network. In classification tasks, we typically stop when the validation error ceases to improve. But in GANs, the Generator and the Discriminator are locked in a *zero-sum game*: as one improves, the other worsens.

The ultimate goal is to reach a **Nash equilibrium**, a point where neither network can improve by changing its strategy:

- The **Generator** produces examples so realistic that the Discriminator can no longer distinguish real from fake.
- The **Discriminator** can only guess randomly, achieving at best 50% accuracy.

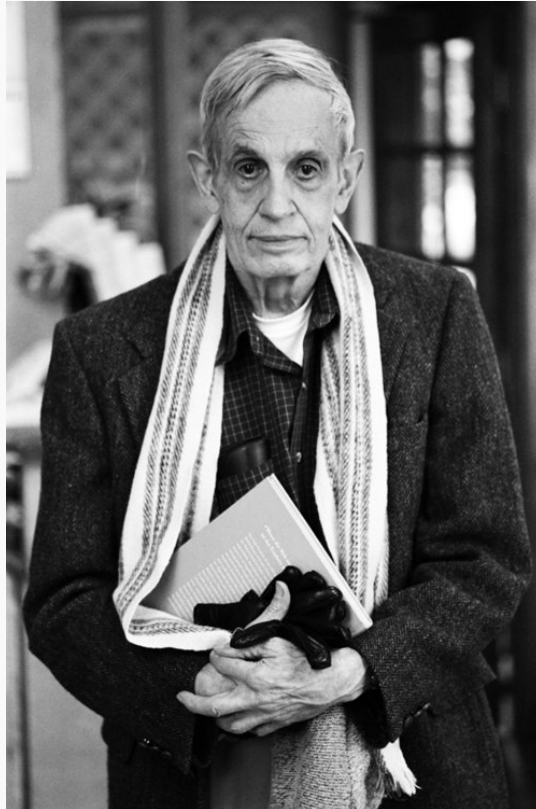
At this point, the GAN is said to have **converged**. However, in practice, finding this equilibrium is extremely challenging. Training often oscillates, diverges, or collapses before stability is reached. Despite these difficulties, GANs have achieved groundbreaking results in generating images, videos, and other modalities—even without guaranteed convergence.

Historical Toolbox: John Nash and Nash Equilibrium

John Forbes Nash Jr. (1928–2015) was an American mathematician and Nobel Prize laureate whose life was famously depicted in *A Beautiful Mind*.

In game theory, Nash proved that in any competitive situation involving multiple players, there exists a state of balance—called a **Nash equilibrium**—where no player can gain by changing their strategy unilaterally.

For GANs, this equilibrium corresponds to the Generator and Discriminator reaching a point where neither can improve without the other being made worse.



This section ties directly to the training loop animation: at the beginning the Discriminator dominates, but over time, the Generator improves until both networks reach a delicate balance.

6 Training Tips and Challenges

Note

Common GAN training challenges:

- **Mode collapse:** Generator produces limited variety of outputs.
- **Non-convergence:** Training oscillates and fails to stabilize.
- **Vanishing gradients:** Discriminator becomes too strong, providing little gradient for generator updates.

Best Practices

- Use proper initialization and normalization (e.g., batch norm)
- Use learning rate schedules
- Employ improved loss functions (WGAN, hinge loss)
- Monitor both generator and discriminator losses

7 Hands-on Lab: Implementing a Simple GAN in PyTorch

Hands-on Lab

Objective: Generate handwritten digits similar to MNIST dataset.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms

# Generator
class Generator(nn.Module):
    def __init__(self, latent_dim):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(True),
            nn.Linear(128, 784),
            nn.Tanh()
        )
    def forward(self, z):
        return self.model(z)

# Discriminator
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(784, 128),
            nn.LeakyReLU(0.2),
            nn.Linear(128, 1),
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.model(x)

# Training loop, loss, and optimizer would follow here...
```

8 Variants of GANs

Example

Popular GAN variants:

- **Conditional GAN (cGAN):** Generator and discriminator conditioned on class labels.
- **Wasserstein GAN (WGAN):** Improves stability using Wasserstein distance.
- **CycleGAN:** Translates images from one domain to another without paired data.
- **StyleGAN:** Generates high-quality photorealistic images with style control.

9 Applications of GANs

Since their invention, GANs have been recognized as one of the most influential ideas in deep learning. Yann LeCun, director of AI research at Facebook, even called GANs *“the coolest idea in deep learning in the last 20 years.”*

Unlike many technical breakthroughs that stay confined to labs, GANs have captured public imagination and received attention from major media outlets such as the New York Times, BBC, and Scientific American.

What makes GANs special?

- **Hyperrealistic imagery:** GANs can synthesize faces that look convincingly real, despite not belonging to any actual person.
- **Image-to-image translation:** GAN variants like CycleGAN can transform a horse into a zebra, or a photograph into a Monet-style painting—without needing paired training data.
- **Applications:** GANs are used in fashion (e.g., Amazon’s AI stylist), in medicine (data augmentation for diagnosis), and even as a stepping stone toward Artificial General Intelligence.

Promise and risk

The creative power of GANs comes with challenges. They enable positive applications in art, science, and industry, but also raise ethical concerns around misinformation, such as fake

videos and fabricated news. As with all transformative technologies, responsible awareness and use are essential.



Figure 4: Hyperrealistic but fake human faces generated by Progressive GANs [?].

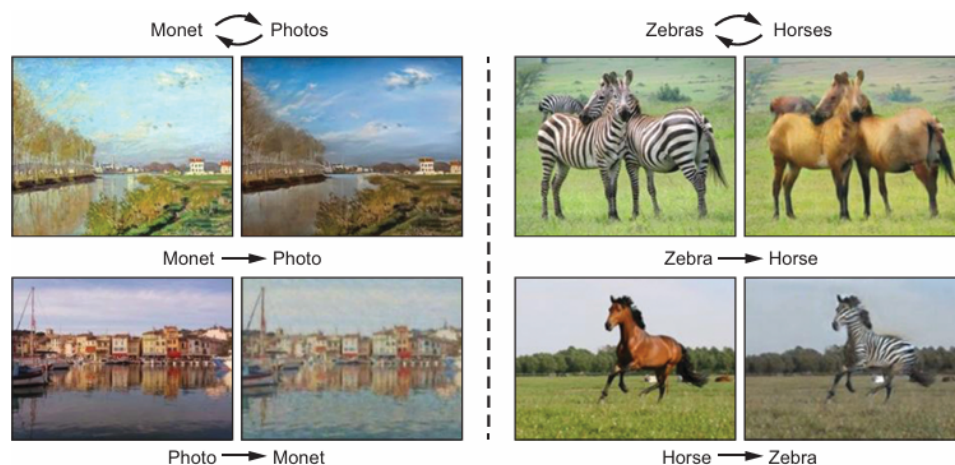


Figure 5: By using a GAN variant called CycleGAN, we can turn a Monet painting into a photograph or turn an image of a zebra into a depiction of a horse, and vice versa. (Source: “Unpaired Image-to-Image Translation Using Cycle-Consistent Adversarial Networks,” by Jun-Yan Zhu et al., 2017).

Example

- Image synthesis: faces, art, fashion
- Super-resolution: upscaling low-res images
- Image-to-image translation: day to night, sketches to photos
- Data augmentation for deep learning models
- Anomaly detection: detecting rare events in images or videos